

Estructura de datos

Tipos de datos abstractos

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2026-II

UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO

- 1 Tipos de datos abstractos
- 2 Elementos de Java: wrappers, genéricos e iteradores
- 3 Implementación de TDA mediante orientación a objetos



Tipo de dato abstracto

Definición

Un **tipo de dato abstracto (TDA)** describe un **conjunto de datos** y las **operaciones** que **pueden realizarse** sobre ellos, sin especificar **cómo se implementan** internamente.

- Define **qué operaciones** están **disponibles**.
- Define el **comportamiento esperado**.
- **Oculto** los **detalles internos** de implementación.
- Permite separar la **especificación** de la **representación**.

Ejemplo

Una lista permite **agregar, eliminar, buscar y recorrer** elementos, sin importar si **internamente** usa arreglos o nodos enlazados.

Especificación de un TDA

Definición

La **especificación** de un TDA **indica** cómo debe **usarse** la estructura, **no** cómo está **construida** internamente.

Operación	Comportamiento esperado
agregar(e)	Inserta un elemento en la lista.
eliminar(i)	Elimina el elemento ubicado en la posición i .
obtener(i)	Regresa el elemento almacenado en la posición i .
tamano()	Regresa el número de elementos almacenados.
estaVacia()	Indica si la estructura no contiene elementos.

Especificación e implementación

Especificación

Describe el TDA desde el **punto de vista** del usuario.

- Nombre de las **operaciones**.
- **Parámetros**.
- **Valores de retorno**.
- **Comportamiento** esperado.
- **Errores** posibles.

Ejemplo

El **TDA lista** puede implementarse mediante un **arreglo redimensionable** o mediante una **lista enlazada**.

Implementación

Describe cómo se construye internamente el TDA.

- Arreglos.
- Nodos.
- Referencias.
- Algoritmos internos.
- Manejo de memoria.



Invariantes de una estructura de datos

Definición

Un **invariante** es una **condición** que debe mantenerse **verdadera** durante **toda la vida** de una estructura de datos.

Ejemplos en una lista

- **size** **nunca** debe ser negativo.
- Si la lista está **vacía**, **size** == 0.
- Los **índices válidos** cumplen:

$$0 \leq index < size$$

- En una lista enlazada, **el último nodo** debe apuntar a **null**.

¡Importante!

Cada operación debe **preservar** los **invariantes** de la estructura.



Clases Wrapper (envoltorio)

Definición

Las **clases Wrapper** permiten representar **tipos primitivos** como **objetos**.

Tipo primitivo	Clase Wrapper
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

Uso de clases Wrapper

```
Integer numero = 10;  
Double promedio = 8.7;  
Character letra = 'A';  
Boolean activo = true;
```

Autoboxing

Java ☞ puede convertir automáticamente un **tipo primitivo** en su clase **Wrapper**.

```
int x = 5;  
  
Integer y = x;           // autoboxing  
int z = y;               // unboxing
```

¡Importante!

Las **estructuras genéricas** en Java **no trabajan directamente** con **tipos primitivos**; por eso se usan clases Wrapper.

Genéricos

Definición

Los **genéricos** permiten definir **clases**, **interfaces** y **métodos** que trabajan con **distintos tipos de datos** sin duplicar código.

- Permiten crear **estructuras reutilizables**.
- Evitan **conversiones manuales** de tipo.
- Mejoran **la seguridad** en tiempo de compilación.
- Son **fundamentales** para **implementar TDA** en **Java** ☕.

Ejemplo

`Lista<String>`

`Lista<Integer>`

`Lista<Estudiante>`

Clase genérica

```
public class Caja<T> {  
    private T contenido;  
  
    public void guardar(T contenido) {  
        this.contenido = contenido;  
    }  
  
    public T obtener() {  
        return contenido;  
    }  
}
```

Parámetro de tipo

El símbolo T representa un **tipo** que será **definido** al momento de **crear el objeto**.

```
Caja<String> cajaTexto = new Caja<>();  
Caja<Integer> cajaNumero = new Caja<>();
```

Parámetros de tipo comunes

Parámetro	Uso frecuente
E	Elemento de una estructura de datos.
T	Tipo genérico general .
K	Clave en estructuras tipo mapa.
V	Valor asociado a una clave.

¡Importante!

Por ejemplo, en `Lista<E>`, el parámetro E representa el **tipo de elementos** almacenados en la lista.

Iteradores

Definición

Un **iterador** es un **objeto** que permite **recorrer los elementos** de una **estructura de datos sin exponer** su **representación interna**.

- Permite **recorrer una estructura** elemento por elemento.
- **Oculto** si la estructura usa **arreglos, nodos** u **otra representación**.
- Facilita el **uso de ciclos** como `while` o `for-each`.
- Se basa en la **interfaz** `Iterator<E>`.

Uso de Iterator<E>

```
Iterator<String> it = nombres.iterator();

while (it.hasNext()) {
    String nombre = it.next();
    System.out.println(nombre);
}
```

Métodos principales

- `hasNext()`: indica **si existe** otro elemento.
- `next()`: **regresa** el siguiente elemento.

¡Importante!

El iterador **controla el recorrido** sin revelar cómo están almacenados los datos.

Iteradores y for-each

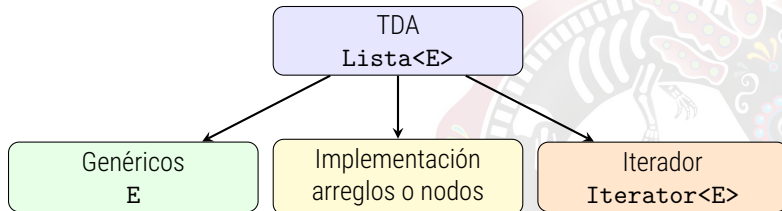
Para que una estructura pueda usarse con `for-each`, debe **implementar** la interfaz `Iterable<E>`.

```
public interface Lista<E> extends Iterable<E> {  
    void agregar(E elemento);  
    E obtener(int indice);  
    int tamano();  
}
```

Entonces puede **recorrerse** así:

```
Lista<String> nombres = new Lista<>();  
  
for (String nombre : nombres) {  
    System.out.println(nombre);  
}
```

Relación entre TDA, genéricos e iteradores



¡Importante!

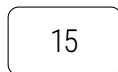
Un **TDA** define las **operaciones**; los **genéricos** permiten **reutilizarlo** con distintos tipos de datos; los **iteradores** permiten **recorrerlo** sin conocer su representación.

TDA Lista

Definición

Una **lista** es un **tipo de dato abstracto (TDA)** que representa una **colección ordenada de elementos**, en la que cada elemento ocupa una **posición** dentro de una secuencia.

primer elemento



0

28

1

34

2

47

3

52

4

último elemento

Características

- Los elementos mantienen un **orden determinado**.
- Cada elemento puede identificarse por su **posición**.
- Se pueden realizar operaciones de **inserción, eliminación, búsqueda, acceso y recorrido**.

Un TDA mediante una interfaz

«interface» Lista
void agregar(E elemento); E obtener(int indice); E eliminar(int indice); int tamano(); boolean estaVacia();

```
public interface Lista<E> {
    void agregar(E elemento);
    E obtener(int indice);
    E eliminar(int indice);
    int tamano();
    boolean estaVacia();
}
```

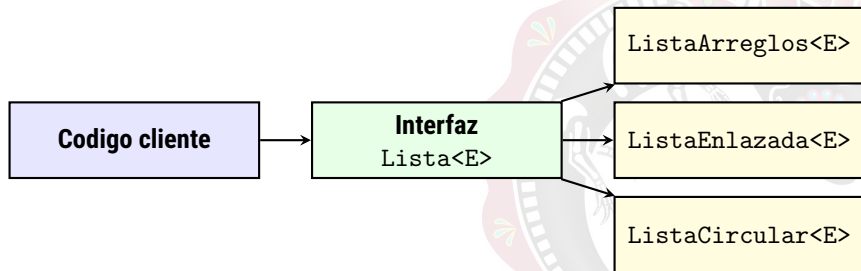
Interpretación

La interfaz `Lista<E>` define el **contrato** que **deberán cumplir** todas sus **implementaciones**.

Implementaciones posibles

`ListaArreglos<E>`, `ListaEnlazada<E>`, `ListaDoblementeEnlazada<E>`
y `ListaCircular<E>`.

Interfaz e implementación



Independencia

El **código cliente** utiliza las operaciones definidas por la **interfaz**, sin depender directamente de la **representación** interna.

Clases concretas

Una **clase concreta** **implementa** las **operaciones definidas** por la interfaz.

```
public class ArrayList<E> implements List<E> {  
    private E[] elements;  
    private int size;  
  
    @Override  
    public void add(E element) {  
        // Implementacion usando arreglos  
    }  
  
    @Override  
    public E get(int index) {  
        // Recuperacion por indice  
        return elements[index];  
    }  
}
```

La interfaz define **qué se puede hacer**; la clase concreta define **cómo se hace**.

Encapsulamiento de la representación

Definición

El **encapsulamiento** consiste en **ocultar** los **detalles internos** de una estructura y **permitir el acceso** únicamente mediante operaciones controladas.

- Los **atributos internos** se declaran como **private**.
- El usuario de la estructura **no accede directamente** a los **datos internos**.
- Las operaciones públicas **preservan** los **invariantes**.
- Permite **cambiar** la **implementación sin cambiar** el **código cliente**.

Ejemplo

Una lista puede usar internamente un arreglo o nodos, pero el usuario solo invoca métodos como `agregar()`, `obtener()` o `eliminar()`.

Excepciones

Las **excepciones** permiten reportar condiciones inválidas durante el uso de una estructura de datos.

```
private void checkIndex(int index) {  
    if (index < 0 || index >= size) {  
        throw new IndexOutOfBoundsException("Indice: " +  
            index + ", tamaño: " + size);  
    }  
}
```

Ejemplos de errores

- Consultar una **posición inválida**.
- **Eliminar** de una estructura vacía.
- **Insertar** en una posición fuera de rango.

Fin de la presentación

¿Preguntas o comentarios?

